Build documentatoin

PROOFPIX - PHASED BUILD DOCUMENTATION

OVERALL ARCHITECTURE (Same as before, for reference)
100% client-side Next.js. 4 pages, 7 engines. No backend.

PHASE 0: Project Foundation - 30 mins

Goal: Empty but deployable skeleton.

Tasks:
1. `npx create-next-app proofpix --typescript --tailwind`
2. Install `crypto-js` and `@types/crypto-js`
3. Create folder structure: `/lib`, `/components`, `/app/encode`, `/app/verify`, `/app/vault`, `/app/lab`
4. Create empty pages with "Coming Soon" text
5. Deploy to Vercel once to get link

Acceptance: `npm run dev` works, Vercel link live.

PROMPT FOR AI - PHASE 0:
> Create Next.js 14 app with Tailwind, TypeScript, shadcn. Create folder structure /lib, /components, /app/encode, /app/verify, /app/vault, /app/lab. Install crypto-js. Each page just shows its name. Make header with navigation: Encode | Verify | Vault | Lab. Minimal black/white design.

PHASE 1: Core Pixel Engine + Basic Encode (MVP Day 1)

Goal: You can upload ONE image and hide text inside it. No encryption yet, just raw hiding. This proves steganography works.

What to build:
1. `lib/pixelEngine.ts` - `loadImageToImageData(file)`, `imageDataToDataURL(data)`, `getCapacity()`
2. `lib/stegoEngine.ts` - ONLY these two functions:
  - `textToBinary(text)` / `binaryToText(binary)`
  - `encodeBasic(original: ImageData, message: string): ImageData` -> Hides message in R channel LSB, first 32 pixels store length
  - `decodeBasic(data: ImageData): string`
3. `components/DropZone.tsx` - Drag drop with preview
4. `/app/encode/page.tsx` - UI: DropZone + Input Owner Name + Button Encode + Canvas Preview + Download Button

What NOT to build in this phase: No encryption, no redundancy, no PSNR, no bulk.

Acceptance Test:
- Upload 1080p PNG, type "Prachi", click Encode, Download.

- Upload that downloaded image to same page's decode function (temporarily add decode button), it should return "Prachi"

PROMPT FOR AI - PHASE 1:
> Phase 1 - Build Core Pixel Engine. Implement lib/pixelEngine.ts with loadImageToImageData using Canvas API and getCapacity. Implement lib/stegoEngine.ts with textToBinary, binaryToText, encodeBasic and decodeBasic using LSB on R channel only. First 32 pixels store length. Build /app/encode/page.tsx with DropZone component, owner name input, Encode button, canvas preview, and download. Keep it simple, no encryption, no redundancy yet. Test: encode then decode same image should return same text.

PHASE 2: Security + Quality Layer

Goal: Make it secure and measurable. This is what makes it resume-worthy.

What to build:
1. `lib/cryptoEngine.ts` - `encrypt(plaintext, passphrase)` and `decrypt(cipher, passphrase)` using AES-256-GCM with PBKDF2 100k iterations. IV random 12 bytes.
2. Update `stegoEngine.ts`:
   - `encodeSecure()` -> calls cryptoEngine.encrypt then encodeBasic
   - `decodeSecure()` -> calls decodeBasic then cryptoEngine.decrypt
   - `calculatePSNR(original, watermarked): number`
3. Update `/encode` page: Add Passphrase input, show PSNR badge (Green if >40dB, Yellow if 35-40, Red if <35), show Unique ID `PX_` + random
4. Build `/verify` page: DropZone + Passphrase input + Result Card (Verified / Not Found)

Acceptance Test:
- Encode with passphrase "1234" -> Download -> Verify with "1234" -> Should show Verified + owner name
- Verify with wrong passphrase "wrong" -> Should show "Failed - Wrong passphrase or not watermarked"
- PSNR for 1080p should be >44dB

PROMPT FOR AI - PHASE 2:
> Phase 2 - Add Security & Quality. Build lib/cryptoEngine.ts with PBKDF2 100k iterations and AES-256-GCM encrypt/decrypt using crypto-js. Update stegoEngine to encodeSecure/decodeSecure that encrypts payload before hiding and decrypts after extracting. Add calculatePSNR function. Update /encode page to include passphrase input, generate PX_ random ID, show PSNR badge. Build /verify page from scratch with DropZone, passphrase input, and result card that shows Verified with owner/date/id or Not Found. Payload format is JSON.stringify({owner, timestamp, id}).

PHASE 3: Redundancy + Error Correction (Robustness Core)

Goal: Make watermark survive crop and compression. This is your engineering USP.

What to build:
1. `lib/redundancyEngine.ts`:
   - `addHamming74()` - 4 bits -> 7 bits with parity, can correct 1 error
   - `applyRedundancy()` - Take binary payload and embed it at 3 offsets: [0, width*height/3,

width*height*2/3]
  - `majorityVote(results: string[])` - Most common among 3 wins
2. Update `stegoEngine.ts` to use redundancyEngine:
  - Encode: Apply Hamming then apply 3x redundancy
  - Decode: Read from 3 offsets, correct Hamming, then majority vote. Return confidence = successful_reads/3 * 100
3. Update Verify page to show Confidence %: "Verified - Confidence 100% (3/3 copies found)" or "66% (2/3 copies)"

Acceptance Test:
- Encode image -> Manually crop 30% in any photo editor -> Verify -> Should still show Verified with 66% confidence
- This test is what you will demo in interview

PROMPT FOR AI - PHASE 3:
> Phase 3 - Add Robustness. Build lib/redundancyEngine.ts with Hamming(7,4) encode/decode that can correct 1-bit error per 7 bits, and applyRedundancy that embeds same payload at 3 offsets in image, and majorityVote. Update stegoEngine encodeSecure to first apply Hamming then embed at 3 offsets. Update decodeSecure to read from 3 offsets, apply Hamming correction, and do majority voting. Update /verify to show confidence percentage based on how many copies recovered. Do not break Phase 1 & 2.

PHASE 4: Bulk Processor + Vault (Solves Your Many Images Problem)

Goal: Handle 50+ images at once and keep record.

What to build:
1. `lib/vaultEngine.ts` - localStorage wrapper:
  - `save(item: VaultItem)` where VaultItem = {originalName, watermarkedId, owner, date, psnr, hash}
  - `getAll()`, `findById(id)`, `exportCSV()`, `clear()`
2. `lib/bulkEngine.ts` - Queue manager using Web Worker to avoid freezing UI:
  - Takes FileList, processes one by one, updates progress callback
  - Returns {success: [], failed: []}
3. Update `/encode` page: Add toggle "Single / Bulk". In Bulk mode: Drop folder, show queue list with progress bar per file, at end show "Download All as ZIP" (use jszip library) + Vault table
4. Build `/vault` page: Table showing all vault items from localStorage with Search, Export CSV, Clear Vault

Libraries to add: `jszip` for ZIP download

Acceptance Test:
- Drop 10 images -> Bulk encode -> Should show 10/10 progress -> Download ZIP with 10 watermarked PNGs -> Vault page shows 10 entries

PROMPT FOR AI - PHASE 4:
> Phase 4 - Bulk & Vault. Install jszip. Build lib/vaultEngine.ts that saves to localStorage key proofpix_vault with schema originalName, watermarkedId, owner, date, psnr. Build lib/bulkEngine.ts that processes FileList sequentially using Web Worker or async loop with progress callback. Update /encode page to support Bulk mode: folder drop, queue UI with progress, final ZIP download via jszip, auto-save to vault. Build /vault page: table of all vault items with search, Export CSV, Clear Vault. Keep Phase 1-3 working.

PHASE 5: Robustness Lab + Polish + Deployment

Goal: Your WOW page for portfolio.

What to build:
1. `lib/attackEngine.ts`:
  - `compress(imageData, quality)` - Simulate JPEG by drawing to canvas with quality 0.7 then reading back
  - `cropCenter(imageData, percent)` - Crop 30% from center
  - `resize(imageData, scale)` - Resize to 50%
  - `blur(imageData)` - Apply blur via canvas filter
2. Build `/lab` page:
  - Left: Show watermarked image
  - Center: 4 buttons [Compress 70%] [Crop 30%] [Resize 50%][Blur]
  - Right: Show attacked image + Auto-run decodeSecure on attacked image + Show Result "Recovered: YES/NO + Confidence"
  - Bottom: Bar chart using simple divs showing Recovery % for each attack
3. Polish: Add README, Privacy page, Footer, Before/After slider, Landing hero
4. Deployment: Final Vercel deploy

Acceptance Test:
- In Lab, encode one image -> Click Compress 70% -> Should still show Recovered YES
- Click Crop 30% -> Should show YES with 66% confidence

PROMPT FOR AI - PHASE 5:
> Phase 5 - Robustness Lab & Polish. Build lib/attackEngine.ts with 4 functions: compress (simulate JPEG quality), cropCenter, resize, blur, all using Canvas API. Build /lab page: Top shows current watermarked image, middle has 4 attack buttons, bottom shows attacked image preview plus auto-verification result using decodeSecure, plus bar chart of recovery rates. Add landing page hero with 2 cards Encode/Verify. Add Privacy page explaining client-side. Polish all pages with shadcn, minimal design. Final Vercel build ready.

HOW TO USE THIS DOC

1. Give AI **Phase 0 prompt** -> Get skeleton
2. Once working, give **Phase 1 prompt + paste Phase 0 code** as context
3. Repeat till Phase 5
4. Never skip phase

Each phase prompt is designed to be given **along with previous phase's code** so AI doesn't break old features.

Want me to start with **Phase 0 + Phase 1 code** right now so you can begin?